DRAFT – please do not distribute

International
Virtual
Observatory
Alliance

IVOA Execution Broker
Version 1.0

IVOA Working Draft 2024-04-25

Working Group
GWS
This version
https://www.ivoa.net/documents/ExecutionBroker/20240425
Latest version
https://www.ivoa.net/documents/ExecutionBroker
Previous versions
This is the first public release
Author(s)
Dave Morris, Sara Bertocco
Editor(s)
Dave Morris

DRAFT – please do not distribute

Abstract

One of the long term goals of the IVOA has been to enable users to move the code to the data. This is becoming more and more important as the size and complexity of the datasets available in the virtual observatory increases.

The IVOA *Execution Broker* provides a step towards making this possible.

The IVOA *Execution Broker* is designed to address a specific question; given an executable thing, e.g. a Python program or Jupyter notebook. What facilities are available to run it?

To do this, the IVOA *Execution Broker* specification defines a data model and web service API for describing executable things and the resources needed to execute them.

Together these components enable a user to ask a simple question *"Where (and when) can I execute my program?"*

This in turn enables users to move code between science platforms. Allowing them to develop their code on one platform and then apply it to a different dataset by sending it to execute on another platform.

Status of this document

This is an IVOA Working Draft for review by IVOA members and other interested parties. It is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use IVOA Working Drafts as reference materials or to cite them as other than "work in progress".

A list of current IVOA Recommendations and other technical documents can be found at https://www.ivoa.net/documents/.

Contents

1 Introduction **5**
- 1.1 Role within the VO Architecture 6
- 1.2 Supplementary documents 6
  - 1.2.1 OpenAPI specification 6
  - 1.2.2 IVOA profiles 7
  - 1.2.3 IVOA Single-Sign-On 8
- 1.3 Executable things 8

DRAFT – please do not distribute

2 Service interaction **9**
2.1 Discovery services . . . 9
2.1.1 Software discovery . . . 11
2.1.2 Data discovery . . . 12
2.2 Execution Broker . . . 12
2.2.1 OfferSet request . . . 13
2.2.2 Resource allocation . . . 14
2.2.3 Execution scheduling . . . 15
2.2.4 Offerset response . . . 16
2.2.5 Session lifecycle . . . 16
2.2.6 Session execution . . . 18
2.2.7 Update options . . . 19

3 The data model **20**
3.1 The *executable* . . . 20
3.1.1 Docker container . . . 22
3.1.2 Jupyter notebook . . . 23
3.2 Resources . . . 23
3.2.1 Compute resources . . . 23
3.2.2 Storage resources . . . 25
3.3 Date and time . . . 31
3.4 Metadata roles . . . 34
3.4.1 The developer . . . 35
3.4.2 The packager . . . 36
3.4.3 The publisher . . . 37
3.4.4 The end user . . . 37

4 Federated architecture **39**

5 Service specification **41**
5.1 General requirements . . . 41
5.2 ExecutionBroker . . . 41
5.2.1 Request method . . . 41
5.2.2 Offer GET . . . 43
5.2.3 Offer POST . . . 44

DRAFT – please do not distribute

6 The data model **46**
6.1 Type URIs and specifications 46
6.2 *ExecutionBroker* request 47
6.2.1 Executable 47
6.2.2 Jupyter notebook 48
6.2.3 Zeppelin notebook 48
6.2.4 Docker container 49
6.2.5 Singularity container 50
6.2.6 Resources 51
6.2.7 Compute resources 51
6.2.8 Generic compute resources 52
6.2.9 Storage resources 54
6.2.10 Authentication 54
6.2.11 DateTime 54
6.3 Response 54
6.3.1 Positive response 54
6.3.2 Offer 54
6.3.3 Negative response 55
6.4 Session 55
6.5 Extensions 56
6.5.1 nvidia gpu 56
6.5.2 Xilinx FPGA 56

A Changes from Previous Versions **57**

A Resource type URLs **58**

References **58**

Acknowledgments

The authors would like to thank all the participants in the IVOA and ESCAPE projects who have contributed their ideas, critical reviews, and suggestions to this document.

Conformance-related definitions

The words “MUST”, “SHALL”, “SHOULD”, “MAY”, “RECOMMENDED”, and “OPTIONAL” (in upper or lower case) used in this document are to be interpreted as described in IETF standard RFC2119 (Bradner, 1997).

DRAFT – please do not distribute

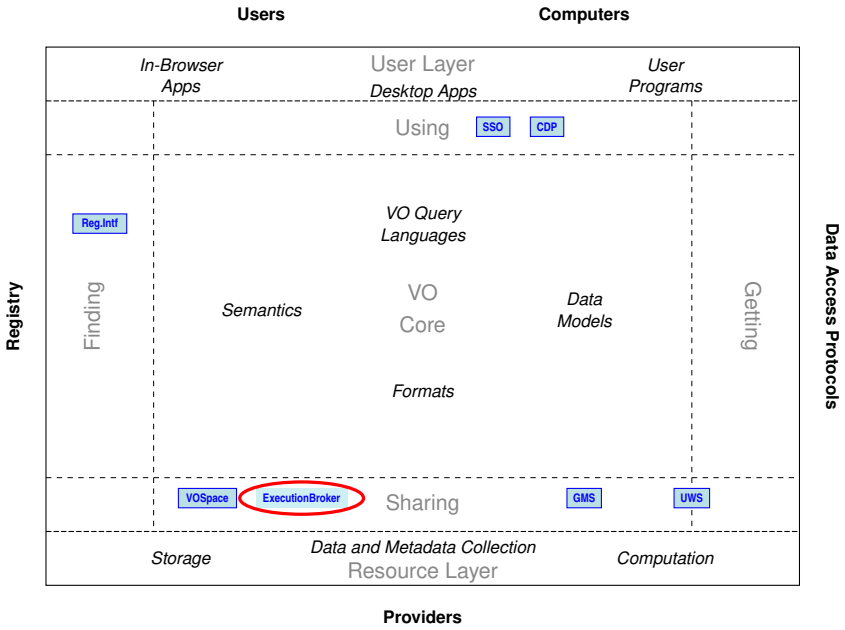


Figure 1: Architecture diagram showing the IVOA *Execution Broker*'s role in the IVOA

The *Virtual Observatory (VO)* is a general term for a collection of federated resources that can be used to conduct astronomical research, education, and outreach. The International Virtual Observatory Alliance (IVOA) is a global collaboration of separately funded projects to develop standards and infrastructure that enable VO applications.

1 Introduction

The IVOA *Execution Broker* specification defines a data model for describing executable tasks and a web service interface for managing them. Together these provide a common interface for service discovery, resource allocation and execution scheduling across a heterogeneous federation of different types of execution platform.

- *ExecutionBroker* data model – a data model for describing execution sessions and their resource requirements.
- *ExecutionBroker* web service – a REST based web service to find execution platforms, allocate resources and schedule execution sessions.

DRAFT – please do not distribute

1.1 Role within the VO Architecture

The IVOA Architecture(Arviset and Gaudet et al., 2010) provides a high-level view of how IVOA standards work together to connect users and applications with providers of data and services. Fig. 1 shows the role the IVOA *Execution Broker* plays within this architecture.

In response to the increasing size and complexity of the next generation of science datasets a number of IVOA members are developing intergrated science platforms which bring together the datasets co-located with the compute resources needed to analyse them.[1][2]

These science platforms make extensive use of the IVOA data models and vocabularies to describe their datasets, and use the IVOA data access services to find and access data from other data providers. In addition, some of the science platforms use IVOA VOSpace services to manage data transfers to and from local storage co-located with the compute resources.

However, to date the IVOA does not provide any APIs or services that enable science platforms to exchange the software used to analyse the data. The IVOA *Execution Broker* provides a step towards making this possible.

This places the IVOA *Execution Broker* in the same region of the IVOA architecture as the IVOA VOSpace specification (Graham and Morris et al., 2009), providing an infrastructure level service that enables service discovery, resource allocation and execution scheduling across a heterogeneous federation of execution platforms.

1.2 Supplementary documents

1.2.1 OpenAPI specification

This document is designed to read in combination with the OpenAPI [3] [4] specification published alongside this document.

The OpenAPI specification defines the technical details of the *Execution Broker* web service interface and the schema for the *Execution Broker* data model. This document compliments the technical specification by using use cases and examples to describe the intended service behaviour and explain the reasoning behind some of the design choices.

The two documents are intended to be read together. The machine-readable OpenAPI spcification defining the *what*, and the human-readable text document describing the *why*.

The OpenAPI specification associated with this document is published in the following files:

[1] https://data.lsst.cloud/
[2] https://rsp.lsst.io/
[3] https://www.openapis.org/
[4] https://swagger.io/specification/

DRAFT – please do not distribute

- `openapi.yaml` - The main service specification, including the *Execution Broker* service API and the core data model.
- `messages.yaml` - a supplimentary data model for INFO, WARN, and DEBUG messages embedded in the service responses.
- `utils.yaml` - a supplimentary data model for re-usable components such as ISO date formats, min/max pairs, and name-value maps.

This text document may include small examples of OpenAPI schema to explain specific points, but these are for information only. Unless otherwise specified, the OpenAPI specification itself should be assumed to be the definitive source and this text document should be considered as secondary.

1.2.2 IVOA profiles

This specification refers to the following documents to cover the details of how an *Execution Broker* web service should use the following protocols and data formats:

- The IVOA REST profile, describing a common pattern for IVOA web services that implement REST[5] webservices.
- The IVOA HTTP profile, describing how IVOA web services should use aspects of the HTTP protocol, including HTTP parameters, message content, content negotiation, and HTTP return codes.
- The IVOA error messages profile, describing a common format for IVOA web services to format error messages.
- The IVOA JSON profile, describing how IVOA web services should serialize message content using the JSON[6] data format.
- The IVOA YAML profile, describing how IVOA web services should serialize message content using the YAML[7] data format.
- The IVOA XML profile, describing how IVOA web services should serialize message content using the XML[8] data format.

Unless otherwise specified, an *Execution Broker* web service implementation should follow the guidelines outined in these documents.

[5] https://en.wikipedia.org/wiki/REST
[6] https://www.json.org/json-en.html
[7] https://yaml.org/
[8] https://www.w3.org/XML/

DRAFT – please do not distribute

1.2.3 IVOA Single-Sign-On

An IVOA *Execution Broker* service MAY use parts of the IVOA Single-Sign-On standard(Taffoni and Schaaf et al., 2017) for authentication, and the IVOA Credential Delegation Protocol (Plante and Graham et al., 2010) for delegating credentials to other services.

1.3 Executable things

To understand the problem that the IVOA *Execution Broker* is trying to solve it is useful to describe what an *executable thing* is in this context. In general terms, this document refers to something that can be executed, or run, as an *executable*.

To explain what this means we can start with a science domain function that we want to perform. For example, the mathematical concept of the square root of a number. We can calculate the square root of a positive number using the Newton–Raphson algorithm[9] which produces successively closer approximations to the result. However, in general case, this mathematical description of the algorithm would not be considered to be an *executable thing*.

We can write a Python program to use this algorithm to calculate the square root of a number. This is the first identifiable *executable thing* in our example. To be able to use this *executable thing*, you would need a computing resource with the appropriate hardware and software environment. In this case, a computing resource with the Python interpreter installed along with the additional Python modules required by the program. This environment is often referred to as the Python runtime.

In the context of science platforms and data science, a common pattern is to provide this environment using a Docker[10] or OCI[11] container to package the Python program and Python runtime together as a single binary object. This package, or container, is itself an *executable thing*. One which requires a different execution environment than the original Python program. The aim of containerization is to package software components together with all the libraries and dependencies they need as a single binary object that interfaces with a standard execution environment, referred to as the *container runtime*. To be able to use this *executable thing*, you would need a computing resource with the appropriate hardware and software environment. In this case, a computing resource with the Docker or OCI container runtime installed.

We could also create a Jupyter notebook that demonstrates how to use our Python program. This is the third *executable thing* in our example. One which provides an interactive environment for the user to experiment with.

[9] https://en.wikipedia.org/wiki/Newton%27s_method
[10] https://docs.docker.com/get-started/what-is-a-container/
[11] https://opencontainers.org/

DRAFT – please do not distribute

As before, to be able to use this *executable thing*, we would need a computing resource with the appropriate hardware and software environment. In this case, a computer with the Jupyter notebook platform installed along with all the Python modules needed by our Python program. In the context of science platforms and data science, a common pattern is to provide this environment as a web service that allows the user to interact with the Jupyter notebook via a web browser.

From one algorithm that implements a science domain function, we have created three different *executable things*. A Python program, a Docker container packaging the Python program, and an interactive Jupyter notebook that demonstrates how to use the Python program. Each of which requires a different computing environment to execute. A basic Python runtime, the Docker runtime, and a Jupyter notebook service.

We may also want to consider the data that we are applying the algorithm to and the compute resources that will be needed to process it. If we are running some small experiments to learn how to use the algorithm, then a basic computing resource will probably be sufficient. However, if we have a dataset of ten million numbers that we want to process, then we may need to consider adding extra storage to handle the input data and the results. For a large dataset it may also be worth using a GPU to accelerate the calculation.

The IVOA *Execution Broker* data model provides a way to describe what each of these *executable things* are and what resources are needed to execute them. This can include things like number of CPU cores and amount of memory it needs, whether it needs a GPU, the location of the input data, the storage space needed to perform the calculation, and the storage space needed to save the results.

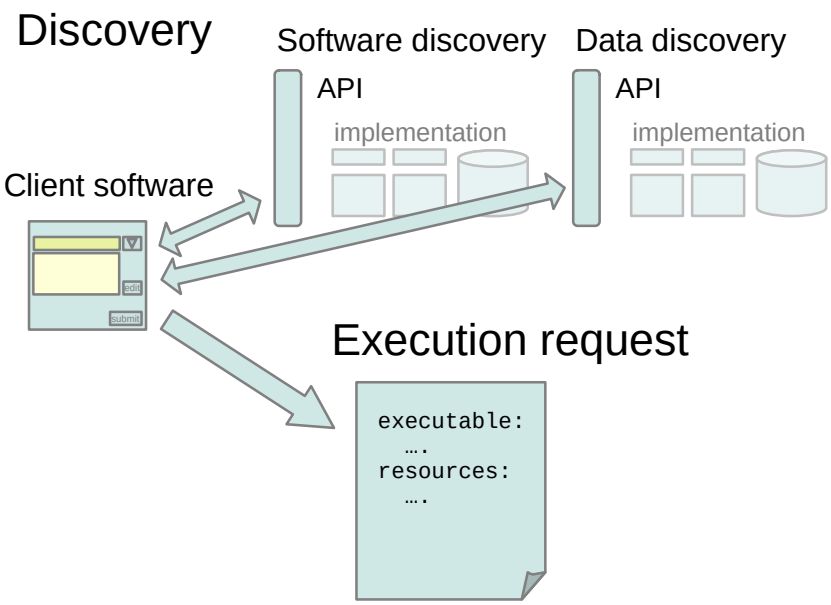
2 Service interaction

The interaction between a user, the client application they are using, and the services available in the Virtual Observatory can be described as a conversation to discover where, how, and when, an *executable thing* that the user has chosen can be executed.

2.1 Discovery services

The conversation starts at the discovery stage, where the user uses discovery services to select the software and datasets that they want to work with.

DRAFT – please do not distribute



The detailed specification for the software and data discovery services are beyond the scope of this document. However we can outline some general requirements for them.

In both cases, the discovery process should not depend on the technical details of the software or the datasets, but on their science domain functionality and properties.

From a science user's perspective they want to be able to find software that implements a particular clustering algorithm, or a dataset that is indexed according to a particular coordinate system. The programming language the software is written in and the file format of the dataset are at best secondary criteria. In our square root example, we would expect our user to use search terms like *'square root'* or *'newton raphson'* to find the software they need. We wouldn't expect them to start out looking for a *'python'* or *'docker'* as their key search terms. Ideally, if the *Execution Broker* service functions as intended, a science user should not need to know about programming languages, software packaging or file formats. The *Execution Broker* service should hide as much as possible of the technical details, enabling the science user to get on with science.

Another important consideration is these discovery services should be designed to be domain agnostic. Meaning that it should be possible to swap out an astronomy based discovery service for an equivalent biochemistry discovery service and although the domain specific terms and vocabulary will be different, the techical details of the service interfaces should be the same.

DRAFT – please do not distribute

2.1.1 Software discovery

There are three main components involved in software discovery, the metadata schema for describing the software, one or more search services, and the repositories where the *executable things* are stored.

The vocabularies and schema need to be based on use cases that start by describing what the scientist wants to do, and from that derrive what software tools they would need, and what terms they would naturally use to describe them.

The IVOA semantics and data modelling working groups have a lot of experience developing vocabularies and data models to descibe science data products, and is well placed to develop the vocabularies needed to descibe astronomy software.

It is important to keep in mind that the requirement is not to model the technical properties of the software itself e.g. what programming language it is written in or who funded the development. The important things to model are the search terms that a scientist is most likley to use to try to find the software they need.

The second component is a searchable database that acceopts a list of search terms and responds with a list of *metadata documents* that describe *executable things* that match the criteria.

Before we look in detail at the content of the *metadata documents* it is worth looking at where the *metadata documents* are stored in relation to the search service and the repositories where the *executable things* are stored.

In one scenario, all of the components can be co-located by the same service. The database of search terms, the *metadata documents*, and the binary files containing the *executable things* can all be hosted by the same service implementation.

TODO diagram

An alternative implementation could store them at different locations, using existing off-the-shelf software and services to host them.

There are a number of widely available content managment systems, both commercial and open source, that would be capable of implementing the database of search terms.

If the *metadata documents* are stored in the same database, then the response from a database search could contain the *metadata documents* themselves.

TODO diagram database, results, contain *metadata documents* from database

Alternatively, the *metadata documents* could be stored at a separate location, in an online git repository for example, and the database search response simply contains a list of URLs that point to the individual *metadata documents*.

DRAFT – please do not distribute

TODO diagram database, results, links to *metadata documents* in external repositories

The third part of the set is the binary image of the *executable thing*. In most cases it would probably make sense for the *metadata document* to reference the *executable thing* as a binary file stored in an external repository rather than trying to include the *executable thing* as a binary blob in the database.

TODO diagram database, results, links to *metadata documents* with links to images

The system can use standard cryptographic signatures and checksums to ensure the validity of the *metadata documents* and the binary images they refer to even when they are stored and accesed via external 3rd party services.

In summary, there are two things that need to be standardised for a software discovery service:

- The inputs to the discovery service, including the metadata vocabularies used to describe the software in terms that make sense to the scientist looking for them. For example what algorithm it implements, the type of input data it operates on, and the type of results it generates.
- The outputs of the discovery service, including the *metadata documents* defined by this specification, that describe the binary images that package the software as *executable things*.

The other components in the software discovery stack, the database of search terms, and the storage and access services for the *metadata documents* and binary images, do not need to be standardised at this stage.

2.1.2 Data discovery

TODO - update this with reference to IVOA data product type and how this links on the executable parameters. https://www.ivoa.net/rdf/product-type/2024-05-19/product-type.html

2.2 Execution Broker

Once the user has identified the software and data resources that they want to use, the client application brings together details of the *executable thing* the user has selected, the compute, storage, and data resources they want to use, along with a schedule describing when the user wants it to execute, to create a *metadata document* description of the *ExecutionBroker session* the user wants to execute.

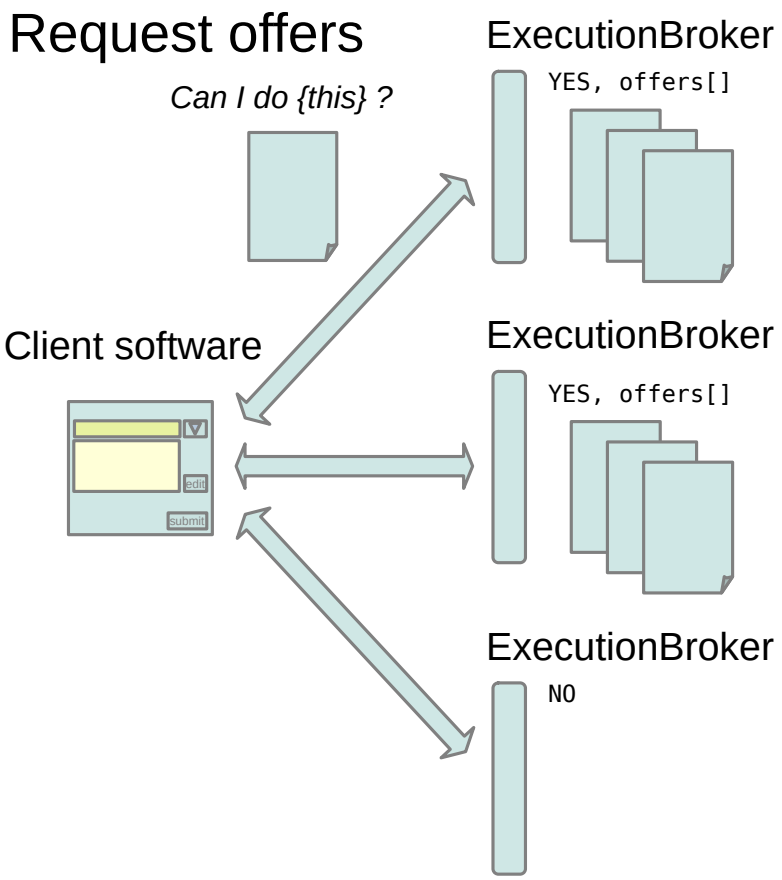
DRAFT – please do not distribute

```
# ExecutionBroker OfferSet request.
executable:
  ....
resources:
  ....
schedule:
  ....
```

2.2.1 OfferSet request

The client sends the *metadata document* description to one or more *ExecutionBroker* services to ask if they can meet the requirements and execute the *ExecutionBroker session*.

Each *ExecutionBroker service* evaluates the request and responds with a top level `YES|NO` answer, and if the answer is `YES`, a list of one or more *ExecutionBroker offers* describing how the requested *ExecutionBroker session* could be executed on the platform(s) represented by that *ExecutionBroker service*.



DRAFT – please do not distribute

2.2.2 Resource allocation

The *ExecutionBroker service* is responsible for managing the resources within the underlying *execution platform* that it represents.

The *ExecutionBroker service* is responsible for keeping track of reservations for the resources that it grants to clients in response to execution requests.

The details of how an *ExecutionBroker service* implements this is beyond the scope of this specification.

This specification defines a set of black-box behaviours that an *ExecutionBroker service* MUST implement.

- Each *ExecutionBroker offer* MUST meet the minimum criteria specified in the request.
- Each *ExecutionBroker offer* MUST be within the capabilities of the underlying *execution platform* to be able to provide.
- An *ExecutionBroker service* MAY offer more resources than the minimum value specified in the request.
- An *ExecutionBroker service* MAY offer more execution time beyond the minimum value specified in the request.
- An *ExecutionBroker service* SHOULD NOT offer more resources than the maximum value specified in the request.
- An *ExecutionBroker service* SHOULD NOT offer more execution time beyond the maximum value specified in the request.

This specification recognises a set of black-box behaviours that an *ExecutionBroker service* SHOULD implement.

- An *ExecutionBroker offer* made by an *ExecutionBroker service* is assumed to be made in good faith, with a reasonable probability of success.
- An *ExecutionBroker service* is free to just say `NO` to a request, for whatever reason.

In addition, this specification recognises a number of situational and dispositional factors that may influence how an *ExecutionBroker service* allocates resources.

- Different implementations may represent a range of different types of *execution platform*s.

DRAFT – please do not distribute

  - Science platforms.
    * CANFAR.
    * Azimuth.
    * SciServer.
  - Cloud platforms.
    * Kubernetes platforms.
    * Openstack platforms.
    * Commercial cloud platforms.
  - Service platforms.
    * Slurm services.

- Different implementations may be connected to a range of different sizes of *execution platforms*.
  - Large scale HPC systems.
  - Large scale commercial platforms.
  - Medium scale on-premises platforms.
  - Small scale specialist platforms.

- Different deployments may apply different algorithms to manage the allocation of resources.
  - Maximise the amount of resources offered to users.
  - Maximise the number of jobs a *execution platform* can execute.
  - Prioritise interactive sessions by offering precise start times.
  - Prioritise batch processing by offering wide start ranges.

- Different deployments may apply different policies to manage the allocation of resources.
  - Required security criteria.
  - Recognised identity providers.
  - Required group membership policies.
  - Local resource allocation policies.
  - Local white lists of allowed executables.

2.2.3 Execution scheduling

This is already covered in a differnt section ..

DRAFT – please do not distribute

2.2.4 Offerset response

Each *ExecutionBroker service* will respond to a request for offers with an *ExecutionBroker offerset* containing some metadata about the *ExecutionBroker offerset* itself, followed by a list of *ExecutionBroker offer*s describing how the requested *ExecutionBroker session* could be executed.

Each *ExecutionBroker offer* in the list contains some metadata about the *ExecutionBroker offer* itself, including its UUID identifier and expiry time, followed by details of how and when the *ExecutionBroker session* would be executed.

```
# ExecutionBroker OfferSet response.
result: YES
....
offers:

- uuid: "2e164a1b-7ff6-11ef-8412-4bc36fe2face"
  state: 'OFFERED'
  expires: "2023-09-18T07:05:21"
  ....
  executable:
    ....
  resources:
    ....
  schedule:
    ....

- uuid: "2e16bf4c-7ff6-11ef-8412-4bc36fe2face"
  status: 'OFFERED'
  expires: "2023-09-18T07:05:21"
  ....
  executable:
    ....
  resources:
    ....
  schedule:
    ....
```

2.2.5 Session lifecycle

An *ExecutionBroker session* goes through the following lifecycle stages.

- `OFFERED` The *session* is being offered.
- `ACCEPTED` The *session* has been accepted.
- `REJECTED` The *session* offer has been rejected.
- `EXPIRED` The *session* offer has expired.

DRAFT – please do not distribute

- `WAITING` The *session* is waiting to activate.
- `PREPARING` The resources are being prepared.
- `READY` The *session* is ready to execute.
- `RUNNING` The *session* is executing.
- `RELEASING` The resources are being released.
- `COMPLETED` The *session* has completed.
- `CANCELLED` The *session* has been cancelled.
- `FAILED` The *session* has failed.

The first four `states` are interactive, where the `state` changes are primarily driven by user interactions.

- `OFFERED` The *session* is being offered.
- `ACCEPTED` The *session* has been accepted.
- `REJECTED` The *session* offer has been rejected.
- `EXPIRED` The *session* offer has expired.

When the *ExecutionBroker service* replies with an *ExecutionBroker offerset* containing a list of *ExecutionBroker offer*s, the user can choose to do nothing, to accept one of the *ExecutionBroker offers* from the list that best fits their requirements, or they can reject the *ExecutionBroker offers*.

Each of the *ExecutionBroker offers* in the *ExecutionBroker offerset* represent a temporary reservation for the resources listed in the *ExecutionBroker offer*. This means these resources will not available to other users while the *ExecutionBroker offers* are still valid.

If the user does nothing, then `state` of each of the *ExecutionBroker offers* will automatically be updated to `EXPIRED`, and their associated resources will be released, when their expiry time is reached.

If the user accepts one of the *ExecutionBroker offers* in the *ExecutionBroker offerset* by updating the `state` to `ACCEPTED`, the *ExecutionBroker service* SHOULD update the `state` of the other *ExecutionBroker offers* in the *ExecutionBroker offerset* to `REJECTED` and release the associated resources.

TODO [accept/reject/expire state-transition diagram]

DRAFT – please do not distribute

2.2.6 Session execution

Once the user accepts one of the *ExecutionBroker offers*, the *ExecutionBroker service* takes over and drives the subsequent `state` changes based on the timing schedule and the results of the internal steps within the *ExecutionBroker session*.

- `WAITING` The *session* is waiting to activate.
- `PREPARING` The resources are being prepared.
- `READY` The *session* is ready to execute.
- `RUNNING` The *session* is executing.
- `RELEASING` The resources are being released.
- `COMPLETED` The *session* has completed.
- `CANCELLED` The *session* has been cancelled.
- `FAILED` The *session* has failed.

It is up to the *ExecutionBroker service* to select the right time to change the *session* `phase` from `WAITING` to `PEPARING` and begin preparing the resources so that the *session* is `READY` in time for the `starttime` declared in the *ExecutionBroker offer*.

If it will take 2 hours to transfer the data resources from archive storage to live storage co-located with the compute resources, then the **ExecutionWorker** needs to start the `PREPARING` phase at least 2 hours before the `starttime` declared in the *ExecutionBroker offer*.

Once all the resources are ready, the **ExecutionWorker** changes the *session* `phase` to `READY` to indicate that the all the resources are ready and the *session* is waiting to start.

The **ExecutionWorker** will then wait until the `starttime` declared in the *ExecutionBroker offer* at which point it will start executing the *session* and change the *session* `phase` to `RUNNING`.

When the *session* finishes executing, because the Docker container finished executing, the user closed their Jupyter notebook, or the `maxduration` was reached, the **ExecutionWorker** will change the *session* `phase` to `TEARDOWN` and begin the process of releasing the resources.

If the *session* includes some persistent storage that should last beyond the end of the *session*, then part of the tear-down process may involve transferring results from the *session* onto the persistent storage before the local storage is released.

When the tear-down process completes, the *session* `phase` is changed to `COMPLETED`.

DRAFT – please do not distribute

If an error occurs at any time in the process, the *session* `phase` is changed to `FAILED`. This includes any errors that occur during the tear-down process; for example, because the **ExecutionWorker** was unable to transfer the results onto persistent storage. Then the *session* `phase` is changed to `FAILED`, even if the main part of the execution completed successfully. This is because any workflow steps that follow after this step, will depend not only on the execution being completed, but they also need the tear-down data transfers to complete so that the results from this step are in the right place for the next step to be able to access them.

2.2.7 Update options

Each *ExecutionBroker session* has a unique URL that the client can use to monitor and update its state.

The *ExecutionBroker service* response for an *ExecutionBroker session* includes a list of options that the user may use to update or modify the *ExecutionBroker session*. Which options are available will depend on the current `state` of the *ExecutionBroker session* and the identity and permissions of the authenticated user.

If the *ExecutionBroker session* is still being offered, then the list of available options allow the user to accept or reject the offer by updating the `state` of the *ExecutionBroker session* to `ACCEPTED` or `REJECTED`.

```
uuid: "2e164a1b-7ff6-11ef-8412-4bc36fe2face"
href: "http://..../sessions/2e164a1b-7ff6-11ef-8412-4
   bc36fe2face"
state: 'OFFERED'
expires: "2023-09-18T07:05:21"
....
....
options:
  - type: "uri:enum-value-option"
    path: "state"
    values:
    - "ACCEPTED"
    - "REJECTED"
```

Once the *ExecutionBroker offer* has been accepted and the *ExecutionBroker session* has started to execute, then the list of available options will only allow the user to cancel the execution.

```
uuid: "2e164a1b-7ff6-11ef-8412-4bc36fe2face"
href: "http://..../sessions/2e164a1b-7ff6-11ef-8412-4
   bc36fe2face"
state: 'ACCEPTED'
expires: "2023-09-18T07:05:21"
....
....
```

DRAFT – please do not distribute

```
options:
  - type: "uri:enum-value-option"
    path: "state"
    values:
    - "CANCELLED"
```

Using a dynamic list of options in this way enables the *ExecutionBroker service* to communicate to the client what actions the user is able to take over the lifetime of an *ExecutionBroker session*.

3 The data model

As outlined in section 1.2.1, this section of the document is intended to compliment the OpenAPI specificaton published alongside this document.

The OpenAPI specification defines the technical details of the *Execution Broker* data model. This document uses use cases and examples to describe the reasoning behind some of the design choices.

The two documents are intended to be read together. The machine-readable OpenAPI spcification defining the *what*, and the human-readable text document describing the *why*.

3.1 The *executable*

At the simplest level the client just needs to check whether a platform is able to execute a particular type of *executable* task. For example, *"Is this platform able to run a Jupyter notebook?"*

In order to do this, the request needs to specify the task type, e.g. Jupyter notebook, along with details about it, e.g. where to fetch the notebook from.

The information in this part of the data model will be different for each type of *executable*. Rather than try to model every possible type of *executable* in one large data model, the data model for each type is described in an extension to the core data model.

The data model uses a common pattern for polymorphic [12] types based on a discriminator [13] value to indicate the type of thing it is describing, followed by the specific details for that type.

This is implemented in the OpenAPI specification as an abstract base class containing common fields like a name and UUID identifier, followed by a list of derived types and their type identifiers.

[12] https://swagger.io/docs/specification/v3_0/data-models/inheritance-and-polymorphism/#polymorphism

[13] https://swagger.io/docs/specification/v3_0/data-models/inheritance-and-polymorphism/#discriminator

DRAFT – please do not distribute

```
# OpenAPI schema
AbstractExecutable:
  type: object
  discriminator:
    propertyName: type
    mapping:
      "uri:java-program-1.0":       'JavaProgram'
      "uri:python-program-1.0":     'PythonProgram'
      "uri:docker-container-1.0":   'DockerContainer'
      "uri:singular-container-1.0": 'SingularContainer'
      "uri:jupyter-notebook-1.0":   'JupyterNotebook'
      ....
  properties:
    name:
      description: >
        A human readable name, assigned by the client.
      type: string
    uuid:
      description: >
        A machine readable UUID, assigned by the server.
      type: string
      format: uuid
    type:
      description: >
        The type discriminator.
      type: string
      format: uri
```

The derived types extend this abstract base class to include the details needed to describe their particular type of *executable thing*.

This pattern of using a `type` discriminator to identify the type of thing, followed by the type specific details is used in several places in the *Execution Broker* data model. This enables us to keep the core data model relatively small, defining the common aspects needed to describe an *executable thing* and the resources it needs while allowing the data model to be extended to describe a wide range of different types of things.

This pattern makes it easy for projects outside the core IVOA community to add new types of *executable things* and resources appropriate for their science domain.

Implementations do not need to understand all of the different possible types of *executable*. If a service doesn't recognize a particular type, it can simply reply `NO`.

```
Request  - Can this platform execute <unkown-type> ?
Response - NO
```

DRAFT – please do not distribute

3.1.1 Docker container

The derived type for a Docker container includes properties to describe where to get the Docker image from, including the repository endpoint URL, the name and version tag of the Docker image to download, and a digest checksum for verification.

```
# OpenAPI schema
DockerContainer:
  description: |
    A Docker or OCI container.
    See https://opencontainers.org/
  type: object
  title: DockerContainer
  allOf:
    - $ref: 'AbstractExecutable'
    - type: object
      properties:
        repository:
          description: >
            The image respository URL.
          type: string
        image:
          description: >
            The image name within the repository.
          type: string
        tag:
          type: string
          description: >
            The image version tag.
        digest:
          description: >
            The image digest checksum, used for verification.
          type: string
        ....
```

This results in the following message being sent to request the execution of a Docker container.

```
# ExecutionBroker offerset request.
executable:
  # Common fields from AbstractExecutable
  name: Experiment one
  type: uri:docker-container-1.0

  # The type specific details for a Docker container.
  repository: ghcr.io
  image: ivoa/analytics/Newton-Rahpson-example
  tag: 2024.08.30
  digest: sha256:ac1b....fc70
```

DRAFT – please do not distribute

3.1.2 Jupyter notebook

The data model for a Jupyter notebook needs to describe where to fetch the the notebook from.

```
# ExecutionBroker offerset request.
executable:
  # Common fields from AbstractExecutable
  name: Experiment two
  type: uri:jupyter-notebook-1.0

  # The type specific details for a Jupyter notebook.
  notebook:
    file: https://.../example.jpnb
```

It may also include a reference to a `requirements.txt` file that describes the additional Python modules needed to run the notebook.

```
# ExecutionBroker offerset request.
executable:
  # Common fields from AbstractExecutable
  name: Experiment two
  type: uri:jupyter-notebook-1.0

  # The type specific details for a Jupyter notebook.
  notebook:
    file: https://.../example.jpnb
  requirements:
    file: https://.../requirements.txt
```

3.2 Resources

At the next level the client may want to check whether a platform has sufficient compute resources to execute a particular task. For example, *"Can this platform provide enough resources to run this Jupyter notebook?"*

In order to do this the request would not only need to describe the *executable* itself, but also the minimum level of compute resources needed in terms of CPU cores, memory, and the local storage space needed to execute it.

3.2.1 Compute resources

The data model for describing compute resources is, in most cases, common to all types of *executable*, so the data model for these requirements are defined as part of the core *Execution Broker* data model.

It is important to note that all of the resource requirements are optional. A request to execute a simple Jupyter notebook does not need to include any resource details.

DRAFT – please do not distribute

```
# ExecutionBroker offerset request.
executable:
  type: uri:jupyter-notebook-1.0
  notebook:
    file: https://.../example.jpnb
```

As long as the Jupyter notebook only needs a minimal set of resources to run, then this is all you need to request an *ExecutionBroker session* to execute it.

However, if the Jupyter notebook needs a minimum amount of computing resources to function properly, then the request may include details of a compute resource with the required minimum values.

```
# ExecutionBroker offerset request.
executable:
  type: uri:jupyter-notebook-1.0
  notebook:
    file: https://.../example.jpnb

resources:
  compute:
    - type: uri:generic-compute
      cores:
        requested:
          min: 4
      memory:
        requested:
          min: 16
```

When the *ExecutionBroker service* reponds with a set of offers, it may offer more resources than the minimum specified in the original request. The most common scenario for this is where something like an Openstack *execution platform* can only allocate compute resources from a fixed set of templates or ‘flavors‘ that provide specific amounts of CPU cores, memory, and local storage. In which case, the *ExecutionBroker service* will select the closest match that meets the requested criteria. To support this, the data model includes two sets of values, one for the request and one for the response. This enables a client to clearly display the differences between the request and the offer to the user.

```
# ExecutionBroker offerset response.
executable:
  type: uri:jupyter-notebook-1.0
  notebook:
    file: https://.../example.jpnb
resources:
  compute:
   - type: uri:generic-compute
      cores:
```

DRAFT – please do not distribute

```
        requested:
          min: 4
        offered:
          min: 8
      memory:
        requested:
          min: 16
        offered:
          min: 32
```

3.2.2 Storage resources

The resources section of the data modelcan also be used to specify a range of different storage resources.

There are two main types of storage resources:

- External storage resources that are provided by an external source.
- Internal storage resources that are provided by the platform.

Within the internal storage group there are two types of resources:

- Ephemeral storage available for the duration of the *session*. Created when the *session* starts and released when the *session* is completed.
- Persistent storage that exists beyond the lifetime of the *session*. Created before the *session* starts and remaining after the *session* has completed.

Within the persistent storage group there are another two levels of resources:

- Managed resources that are created and deleted by the platform.
- Unmanaged resources that are created and deleted by an external entity.

The simplest of these are ephemeral storage resources managed by the execution platform. For example, if the Jupyter notebook in our example requires 1024GiB of space to perform its calculations, then this can be specified in the request by defining a simple ephemeral storage resource.

```
# ExecutionBroker offerset request.
request:
  # Details of the executable.
  executable:
    ....
  # Details of the resources needed.
  resources:
```

DRAFT – please do not distribute

```
    ....
    storage:
      - name: scratch-space
        type: uri:ephemeral-storage
        size:
          requested:
            min: 1024
```

To enable the Jupyter notebook to access this storage, we need to add a corresponding `volume` element to the compute resource that describes where to mount the storage resource.

The following request asks for 1024GiB of ephemeral storage that is mounted at `/temp` in the filesystem of the compute resource. The compute volume is linked to the storage resource by the name of the storage resource.

```
# ExecutionBroker offerset request.
request:
  # Details of the executable.
  executable:
    ....
  # Details of the resources needed.
  resources:

    compute:
      - name: compute-001
        ....
        volumes:
          - name: temp-volume
            resource: scratch-space  ----\
            path: /temp                  |
            mode: rw                     |
                                         |
    storage:                             |
      - name: scratch-space         <----/
        type: uri:ephemeral-storage
        size:
          requested:
            min: 1024
```

This pattern of separating the details of how a storage resource is implemented from the details of how it is mounted inside a computing resource is based on a pattern used by Kubernetes to describe storage volumes and their mount points within containers[14].

This pattern can also be used to define a storage resource that imports data from an external source. For example, if the user wanted to use the Jupyter notebook to analyse data stored in an external S3 system, this can

[14] https://kubernetes.io/docs/concepts/storage/volumes/

DRAFT – please do not distribute

be done by defining an external storage resource that describes where the data is, and a corresponding volume mount inside the compute resource.

```
# ExecutionBroker offerset request.
request:
  # Details of the executable.
  executable:
    ....
  # Details of the resources needed.
  resources:

    compute:
      - name: compute-001
        ....
        volumes:
          - name: data-volume
            resource: source-data  ----\
            path: /data                |
            mode: r                    |
                                       |
    storage:                           |
    - name: source-data           <----/
      type: uri:amazon-s3-storage
      endpoint: https://.../echo
      bucket: example-data
```

Note that this request is defining the result the user wants to see; namely the contents of the remote S3 bucket accessible as read-only files in the Jupyter notebook's filesystem. It is not defining how the underlying *execution platform* should go about making it happen.

The *execution platform* could use something like s3fs-fuse[15] to mount the remote S3 bucket as a directory in the Jupyter notebook's filesystem, or it could transfer a copy of the data from the S3 bucket in to local storage. All we are defining here is the result we want to see.

This pattern of separating how data is stored outside the system and how it appears inside a compute resource borrows heavily from the pattern used by Kubernetes to describe persistent volumes[16].

The same pattern can be used to describe a storage resource that can be used to save the analysis results, by defining a persistent storage resource allocated by the system, and a corresponding volume mount inside the compute resource.

```
# ExecutionBroker offerset request.
request:
  # Details of the executable.
  executable:
```

[15] https://github.com/s3fs-fuse/s3fs-fuse
[16] https://kubernetes.io/docs/concepts/storage/persistent-volumes/

DRAFT – please do not distribute

```
    ....
  # Details of the resources needed.
  resources:

    compute:
      - name: compute-001
        ....
        volumes:
          - name: results-volume
            resource: results-storage ----\
            path: /results                |
            mode: rw                      |
                                          |
    storage:                              |
    - name: results-storage          <----/
      type: uri:persistent-storage
      lifecycle: managed
      lifetime:
        requested:
          min: P1D
      size:
        requested:
          min: 10
          units: TiB
```

By setting the storage resource type to `uri:persistent-storage`, and setting the `lifecycle` to `managed`, the client is asking the *ExecutionBroker service* to take care of allocating the storage and managing its lifecycle. It is up to the *ExecutionBroker service* to decide where to store the data and how make it accessible after the *session* has completed.

For example, a science platform may have a Rucio storage system co-located with the *execution platform* which it uses to store user generated data. In which case the *ExecutionBroker service* would respond with an *ExecutionBroker offer* that stores the results in the Rucio system and provides details of how the user can access it after the *session* has completed.

```
# ExecutionBroker offerset request.
request:
  # Details of the executable.
  executable:
    ....
  # Details of the resources needed.
  resources:

    compute:
      - name: compute-001
        ....
        volumes:
          - name: results-volume
```

DRAFT – please do not distribute

```
            resource: results-storage ----\
            path: /results                |
            mode: rw                      |
                                          |
    storage:                              |
      - name: results-storage        <----/
        type: uri:rucio-storage
        lifecycle: managed
        lifetime:
          requested:
            min: P1D
          offered:
            min: P5D
            max: P5D
        size:
          requested:
            min: 22
            max: 22
            units: TiB
        endpoint: http://....
        domain: Project 51
        objectid: cdc78e7d-8032-497e-9a5b-01c720ea2223
```

In this example the client requested at least 10TiB of storage available for 1 day and in response the service is offering up to 22TiB available for 5 days stored in a Rucio system co-located close to the compute platform. It is up to the client to check if it can access the particular Rucio system described in the response before it accepts the *ExecutionBroker offer*.

Offering an *ExecutionBroker offer* with the `lifecycle` set to `managed` and the `lifetime.max` set to `P5D` means that the *ExecutionBroker service* is offering to manage the lifecycle of the storage. The storage will be available until 5 days after the *session* completes and then it will be deleted automatically. The client doesn't need to worry about tidying up afterwards.

It is important to note that at this point in time the storage is proposed, but not yet allocated. The persistent storage is only allocated if the client accepts this particular *ExecutionBroker offer*. This allows an *ExecutionBroker service* to make multiple *ExecutionBroker offers* with different storage options, allowing the client to select and accept the one that best fits its use case.

The same data model can be used the other way around as well. If the client already knows where it wants the data to be stored, for example at a specific VOSpace location, then it can specify this in the request.

```
# ExecutionBroker client request.
request:
  # Details of the executable.
  executable:
    ....
```

DRAFT – please do not distribute

```
  # Details of the resources needed.
  resources:
    ....
    compute:
    - name: "compute-001"
      ....
      volumes:
      - name: "results"
        path: "/results"
        mode: "rw"
    storage:
    - name: "results"
      type: "https://www.purl.org/ivoa.net/storage-types/
   vospace-storage"
      spec:
        endpoint: "http://...."
        path: "/experiment-21/results"
        lifecycle: "unmanaged"
```

It is up to the *ExecutionBroker service* to work out if it is able to access the VOSpace location and mount it as a volume in the compute resource, using either its own authentication, or a delegated form of the authentication provided by the client.

If it can access the VOSpace location, then the *ExecutionBroker* MAY respond with an *ExecutionBroker offer*, otherwise if it can't access the VOSpace location for whatever reason the *ExecutionBroker* MUST respond with `NO`.

Note that in this example, the client has specified the lifecycle as `unmanaged`, which means that the *ExecutionBroker* is not involved in managing the creation or deletion of the data in VOSpace. It is also possible for the client to ask the *ExecutionBroker service* to manage data in an external resource.

```
# ExecutionBroker client request.
request:
  # Details of the executable.
  executable:
    ....
  # Details of the resources needed.
  resources:
    ....
    storage:
    - name: "results"
      type: "https://www.purl.org/ivoa.net/storage-types/
   vospace-storage"
      spec:
        endpoint: "http://...."
        type: "container"
        path: "/experiment-21/results"
        lifecycle: "managed"
        lifetime:
```

DRAFT – please do not distribute

```
          max: "P2D"
```

In this example, the client is specifying an external VOSpace location to store the data, but it is asking the *ExecutionBroker service* to manage the lifecycle, creating the location in VOSpace at the start of the *session* and deleting it 2 days after the *session* completes.

This negotiation of who is responsible for creating and deleting storage resources enables a client to put together a workflow of interconnected steps, with the *ExecutionBroker services* managing the lifecycle of the resources and releasing them automatically after they are no longer needed.

3.3 Date and time

The `datetime` part of the data model enables the client and server to have a conversation about when a *session* can be executed.

The client can specify one or more time periods when it would like to start the *session*, and the minimum duration that it thinks would be needed to complete the *session*.

The *ExecutionBroker service* may respond with one or more *ExecutionBroker offers* that specify when the *session* would start and the maximum duration that the *session* would be allowed to consume. It is then up to the client to select which of the *ExecutionBroker offers* best suits its use case.

The start time for a *session* is expressed as an array of time intervals, as defined by ISO 8601 (International Organization for Standardization, 2004). Specifically, type 1 or type 2 intervals (start/end and start/duration), excluding type 3 and 4 intervals (duration/end and duration only) and excluding repeats[17][18].

Note that the duration part of the interval applies to the start time, specifying a time range during which the *session* may start.

If no duration is specified, this means an absolute start time; e.g. the *session* SHOULD start at 11:30 on 14 August.

```
# ExecutionBroker client request.
....
datetime:
- start: "2023-08-14T11:30Z"
```

If the start and end are specified, this means the *session* SHOULD start somewhere between the start and end values; e.g. the *session* SHOULD start between 11:30 and 12:00 on 14 August.

```
# ExecutionBroker client request.
....
datetime:
- start: "2023-08-14T11:30Z/T12:00Z"
```

[17] https://www.iso.org/iso-8601-date-and-time-format.html
[18] https://en.wikipedia.org/wiki/ISO_8601

DRAFT – please do not distribute

If a duration is specified, this means the *session* SHOULD start somewhere between start and the start plus duration; e.g. the *session* SHOULD start between 11:30 and 12:00 on 14 August.

```
# ExecutionBroker client request.
....
datetime:
- start: "2023-08-14T11:30Z/PT30M"
```

The *ExecutionBroker service* SHOULD respond with one or more *ExecutionBroker offers* that start within the ranges specified in the request. The start times in the *ExecutionBroker offers* MAY be more precise than the start times in the request, but they MUST all occur within at least one of the ranges specified in the request.

The client MAY also specify the maximum and minimum execution duration, expressed as time periods as defined by ISO 8601.

For example, for the unattended batch mode execution of an Docker container, the user might not be concerned about when their *session* starts, but they may want to specify the minimum duration needed to complete the task.

In which case, the client may simply request a minimum duration of 1 hour.

```
# ExecutionBroker client request.
....
datetime:
- duration:
    min: "P1H"
```

The *ExecutionBroker service* MAY respond with *ExecutionBroker offers* that start at different times and set different values for the maximum duration.

It MAY offer a maximum duration of 1 hour in the morning, starting at 11:00.

```
# ExecutionBroker server response.
....
datetime:
- start: "2023-08-14T11:30Z"
  duration:
    min: "P1H"
    max: "P1H"
```

It MAY also offer a longer allocation of 4 hours later in the evening, starting sometime between 22:00 and 23:00.

```
# ExecutionBroker server response.
....
datetime:
- start: "2023-08-14T22:00Z/PT1H"
```

DRAFT – please do not distribute

```
  duration:
    min: "P1H"
    max: "P4H"
```

Different values for start time and duration can be combined with different values for the compute resources to make a range of different *ExecutionBroker offers* to the client.

For example, if a client asks for 2 cores and 2G of memory for 1 hour sometime on 14 August:

```
# ExecutionBroker client request.
  ...
resources:
  compute:
    cores:
      min: 2
    memory:
      min: 2
datetime:
  - start: "2023-08-14/P1D"
    duration:
      min: "P1H"
```

The *ExecutionBroker service* may respond with 2 *ExecutionBroker offers*, the minimum 2 cores and 2G of memory for 1 hour starting at 11:30, and a larger *ExecutionBroker offer* of up to 8 cores and 8Gb of memory for up to 4 hours starting between 22:00 and 23:00.

```
# ExecutionBroker server response.
offers:
- name: "offer-001"
  executable:
    ...
  resources:
    compute:
      cores:
        min: 2
        max: 2
      memory:
        min: 2
        max: 2
  datetime:
    - start: "2023-08-14T11:30Z"
      duration:
        min: "P1H"
        max: "P1H"

- name: "offer-002"
  executable:
    ...
```

DRAFT – please do not distribute

```
resources:
  compute:
    cores:
      min: 2
      max: 8
    memory:
      min: 2
      max: 8
datetime:
  - start: "2023-08-14T22:00Z/T23:00Z"
    duration:
      min: "P1H"
      max: "P4H"
```

It is then up to the client to decide which *ExecutionBroker offer* better suits their use case. Accept the limited *ExecutionBroker offer* in the morning, or accept the more generous *ExecutionBroker offer* with more resources and more time later in the day.

If an *ExecutionBroker service* is offering more than one option for the `datetime` section it MUST make separate *ExecutionBroker offers* for each different option. Even if all of the other parameters are the same, e.g. compute and storage resources, the *ExecutionBroker service* MUST NOT include more than one time slot in the same *ExecutionBroker offer*. Technically the data model allows an array of values for the `datetime` section, but this would impose unnecessary complexity on the client for no real gain in user experience.

3.4 Metadata roles

The full description of an *executable thing* will include several layers of metadata provided by different actors playing different roles within the publishing process.

For our square root example we can identify a number of roles that would each provide layers of the picture nedeed to fully describe an *executable thing*.

The players:

- The developer - The person who wrote the Python program
- The packager - The person who packaged it in a Docker container
- The publisher - The person who published it in a discovery service
- The user agent - The person who wants to use the software

DRAFT – please do not distribute

3.4.1 The developer

The first layer of metadata comes from the person who wrote the software. They have detailed knowledge of what the software does, what execution environment it needs, and what the inputs and outputs are.

For our square root example, it is a Python program which needs a platform with the Python runtime installed, and a list of the Python libraries that the program relies on.

```
executable:
  name: Newton-Raphson example
  type: uri:python-program-1.0
  requirements:
  - numpi: ""
  - astropy: ">= 6.1"
```

The developer also understands how much memory their program needs, whether it can make use of multiple cpu cores, and whether it can make use of a GPU accelerator.

```
resources:
  compute:
    - type: uri:generic-compute
      cores:
        requested:
          min: 4
      memory:
        requested:
          min: 16
      ....
```

The developer also knows about what inputs and outputs the program expects and what file formats can it can handle.

```
executable:
  name: Newton-Raphson example
  type: uri:python-program-1.0
  ....
  parameters:
    - name: INPUT_FILE
      type: uri:filesystem-file
      mode: readonly
      description:
        A table containing a list of numbers to be processed,
        formatted as comma separated text (CSV) or an IVOA
        VOTable.
      formats:
        - type: uri:filetype-text-csv
          ....
        - type: uri:filetype-ivoa-votable
          ....
```

DRAFT – please do not distribute

```
    - name: INPUT_COLUMN
      type: uri:environment-variable
      description:
        The column name within the 'INPUT_FILE' to use.
```

3.4.2 The packager

Although it is possible to publish our square root example as a stand alone Python program, it is not easy to describe the installation process in sufficient detail for it to be automatically deployed on a range of different platforms.

A more portable solution would be to package the Python program in a Docker container, installing and configuring the software along with all of its dependencies inside the container.

This is often done as part of the software development process, but it is a separate step that could be implemented by a different person. To make this distinction clear we can refer to this person or role as *'the packager'*.

This step packages the Python program along with any Python modules it requires, the Python runtime, and any operating system components it requires, into a single standard format binary file, making it much easier to deploy.

The packager would then create a new *metadata document* that describes the *executable thing* as a Docker container rather than a Python program, with details of where to find the Docker image, including the image name, the version tag and an optional checksum.

```
# Executable metadata
executable:
  name: Newton-Raphson example
  type: uri:docker-container-1.0
  repository: ghcr.io
  image: ivoa/analytics/Newton-Raphson-albert
  tag: 2024.08.30
  digest: sha256:ac1b........fc70
  ....
```

Depending on how the software is packaged in the container the new *metadata document* may update the description of the inputs and outputs, and link them to specific locations in the filesystem of a compute resource.

```
# Executable metadata
executable:
  name: Newton-Raphson example
  type: uri:docker-container-1.0
  ....
  parameters:
    - name: INPUT_FILE            <----\
      type: uri:filesystem-file         |
```

DRAFT – please do not distribute

```
      format:                          |
        - type: uri:ivoa-votable       |
          ....                         |
                                       |
resources:                             |
  compute:                             |
    - type: uri:generic-compute        |
      volumes:                         |
        - type: uri:filesystem-mount   |
          parameter: INPUT_FILE     ---/
          filepath: /data/input-data.vot
          mode: readonly
          ....
```

3.4.3 The publisher

This role represents the person who publishes metadata about the software in a discovery service.

Typically changes made at this level would include adding to the description of what the software does, placing it in a particular context to aid discovery, or modifying the execution environment to configure the softweare for a particular domain. This might include things like:

- A project specific discovery service that only includes software vetted by the project. Execution platforms within the project would only accept curated *metadata documents* from that discovery service.
- A domain specific discovery service that modifies the execution environment, configuring the software to analyse a particular type of data.
- A catalog of *metadata documents* maintained as part of a teaching course, modifying the execution environment to integrate the software into the local IT system and setting parameters to configure it to match the course notes.

3.4.4 The end user

The user starts with *metadata document* from the software discovery service and adds some information describing how they want to use the software.

This may include selecting the data resources that they want to use and adding them to the *metadata document* as data resources. This means the *ExecutionBroker service* can include the time needed to transfer the data to local storage before the *ExecutionBroker session* is started.

```
executable:
  ....
resources:
```

DRAFT – please do not distribute

```
  ....
  data:
    - name: input data
      type: uri:simple-data-resource
      location: http:data.example.org/....
      ....
```

Linking the data resources with volumes on the corresponding compute resources enables the *ExecutionBroker service* to mount the data resources at the correct location in the compute resource's filesystem.

```
resources:
  compute:
    - type: uri:generic-compute
      volumes:
        - type: uri:filesystem-mount
          parameter: INPUT_FILE
          resource: input-data          ----\
          filepath: /data/input-data.vot    |
          mode: readonly                    |
          ....                              |
  data:                                     |
    - name: input-data                  <---/
      type: uri:simple-data-resource
      location: http:data.example.org/....
      ....
```

The user can also update the compute resource requirements to reflect how they plan to use the software. In the case of our square root example, the compute resource requirements set by the developers will reflect the original intent of simply demonstrating how to use the Python program.

However, if the user intends to use the software to analyse a much larger dataset they can update the compute resource requirements from 4 cores and 16 GiB of memory to 32 cores and 128 GiB of memory to match their new requirements.

```
resources:
  compute:
  - type: uri:generic-compute
    cores:
      requested:
        min: 32
    memory:
      requested:
        min: 128
```

The user can use the `schedule` section of the *metadata document* to describe when they want the *ExecutionBroker session* to start.

This is covered in more detail in section 3.3 of this document. TODO - add more detail here

DRAFT – please do not distribute

4 Federated architecture

The *ExecutionBroker service* is designed to be used at multiple levels within an organization.

At the low level, an *ExecutionBroker* serice may be implemented as a single web-application linked to a simple Docker execution service. The configuration may be hard coded to only accept a fixed white list of container images and a fixed allocation of compute resources for each job. e.g. the platform will only run a fixed set of container images, and only provides 2 CPU cores, 2G of memory and a maximum 1HR execution time.

A project or organization may deploy several of these low level services, providing a range of different capabilities.

Given a new *session* to execute, a client can simply poll each of the services to see which ones are able to execute it.

The client does not need to have any prior knowledge about the services apart from their endpoint address. This information could come from the IVOA Registry, or it could simply be provided as a flat list in a configuration file for the client.

A more flexible architecture could add another *ExecutionBroker service* a level above the low level task specific services. This service would be configured with a list of the local task specific services and act as an aggregator service sitting between the client and the low level services. This aggregator service would forward a copy of each request it receives to each of the low level services in its list and then aggregate the *ExecutionBroker offers* from the individual responses into a single response that is sent back to the client.

A simple aggregator service would just forward every request to all the low level services below it, regardless of what the request contained.

A more complex aggregator service may have some prior knowledge about what types of task or compute resources each of the lower level services are able to provide, enabling it to make more informed decisions about which low level serviecs to send the requests to.

The aggregator service may also have an understanding of the location of datasets within the organization and be able to route requests to different low level services depending on which datasets were required.

The aggregator service may expose the individual low level *ExecutionBroker* endpoints in its responses, or it may implement a proxy interface hiding the individual services.

This configuration could be used to provide an *ExecutionBroker service* interface that bridges a firewall. Providing a public interface for external clients and forwarding the requests to internal *ExecutionBroker services* that are not accessible from outside the firewall.

A large organization with multiple sites could also deploy a single high level *ExecutionBroker* interface that handles requests for the whole organi-

DRAFT – please do not distribute

zation, forwarding individual requests to mid-level *ExecutionBroker services* at each site which in turn forwards the requests to individual low-level *ExecutionBroker services* for each platform within the local sites.

The implementation at each level may be different, providing different levels of routing and aggregation based on internal knowledge of the capabilities of the level below, but the *ExecutionBroker* interfaces would all be the same.

This could be expanded to include another level of *ExecutionBroker services* that cross organization bondaries, providing a gateway that allows users from one project to access services from other projects and organizations. This gateway service may also provide an authentication translation layer between external public accounts and internal project specific identities and authentication methods.

DRAFT – please do not distribute

5 Service specification

Details of the request and response messages. Timeline of request, offer and accept.

5.1 General requirements

- ....
- ....
- ....

5.2 ExecutionBroker

The main *ExecutionBroker* web service implements two interfaces; a `POST` method for sending requests, and a `GET/POST` interface for accessing *ExecutionBroker offers* and updating their status.

5.2.1 Request method

The main *ExecutionBroker* web service method is a `POST` web service that accepts a `request` as defined in section 6.2 of the data model.

An *ExecutionBroker* web service MUST be able to accept both YAML or JSON serializations of the `request`. The serialization SHOULD be declared in the HTTP request using the `Content-Type` header.

- `application/yaml` for YAML.
- `application/json` for JSON.
- If the serialization is not declared, the default is assumed to be YAML.
- If the both YAML and JSON are declared, the default is assumed to be YAML.

```
POST /request HTTP/1.1
Host: foo.example.org
Content-Type: application/yaml

request:
  executable:
    ....
  resources:
    ....
```

DRAFT – please do not distribute

```
POST /request HTTP/1.1
Host: foo.example.org
Content-Type: application/json

"request": {
  "executable": {
    ....
    }
  "resources": {
    ....
    }
  }
```

The *ExecutionBroker service* will evaluate the request and respond with either a positive or negative `response` depending on whether the platform is able to execute the task descibed in the `request`.

An *ExecutionBroker service* MUST be able to provide both YAML and JSON serializations of the `response`. The serialization is determined by the `Accept` header in the HTTP request.

- `application/yaml` for YAML.
- `application/json` for JSON.
- If the serialization is not declared, the default is assumed to be YAML.
- If the both YAML and JSON are declared, the default is assumed to be YAML.

If the platform IS able to execute the task, then the web service will respond with a HTTP response code `200` and a positive `response` content as defined in section 6.3.1.

```
HTTP/1.1 200 OK
Content-Type: application/yaml; charset=utf-8

response:
  result: 'YES'
  offers:
    ....
```

If the platform IS NOT able to execute the task, then the web service will respond with a HTTP response code `200` and a negative `response` content as defined in section 6.3.3.

```
HTTP/1.1 200 OK
Content-Type: application/yaml; charset=utf-8

response:
  result: 'NO'
```

DRAFT – please do not distribute

```
  reasons:
    ....
```

The `request` method is NOT idempotent. Sending the same `POST` request twice MAY return different results.

- The list of *ExecutionBroker offers* in a response depends on the resources available at the point when the request is made.
- Sending a request generates a new list of *ExecutionBroker offers*. Those *ExecutionBroker offers* will contain references to resources that are reserved for the lifetime of the `offer`.
- If a second request is made soon after the first request, then the *ExecutionBroker offers* made in response to the first request may still be active. In which case, the resources listed in those *ExecutionBroker offers* may still be reserved, reducing the resources available for the second request.
- If a second request is made some time after the first, then other users may have requested and accepted *ExecutionBroker offers* reducing the resources available for the second request, or the platform may have completed some tasks, increasing the resources available for the second request.

5.2.2 Offer GET

The `offer` `GET` method requests the current state of an *ExecutionBroker offer*. The identifier for the *ExecutionBroker offer* is encoded in the URL of the HTTP request by appending the identifier to the path of the HTP request URL.

```
http[s]://foo.example.org/offers/{identifier}
```

The data model for the content of the response is described in section 6.3.2.

An *ExecutionBroker service* MUST be able to provide both YAML and JSON serializations of the `offer`. The serialization is determined by the `Accept` header in the HTTP request.

- `application/yaml` for YAML.
- `application/json` for JSON.
- If the serialization is not declared, the default is assumed to be YAML.
- If the both YAML and JSON are declared, the default is assumed to be YAML.

DRAFT – please do not distribute

```
GET /offers/2c89f536-3fff-48f7-943f-bcc5c3225be7 HTTP/1.1
Host: foo.example.org
Accept: application/yaml
```

```
HTTP/1.1 200 OK
Content-Type: application/yaml; charset=utf-8

offer:
  ident: "2c89f536-3fff-48f7-943f-bcc5c3225be7"
  status: 'OFFERED'
  expires: "2023-09-18T07:05:21"
  request:
    executable:
      ....
    resources:
      ....
```

5.2.3 Offer POST

The `offer` `POST` method provides the client with a method for updating the status of an `offer`.

The identifier for the *ExecutionBroker offer* is encoded in the URL of the HTTP request by appending the identifier to the path of the HTP request URL.

```
http[s]://foo.example.org/offers/{identifier}
```

The content of the `POST` method should contain the properties of the `offer` that the client wants to change, as defined by the `offer` data model in section **??**.

- Currently, only the `status` of an `offer` is modifiable.
- The `POST` data MAY contain the `offer` `ident`.
  - If the `offer` `ident` is presnet it MUST match the identifier used in the URL.

An *ExecutionBroker* web service MUST be able to accept both YAML or JSON serializations of the `offer`. The serialization SHOULD be declared in the HTTP request using the `Content-Type` header.

- `application/yaml` for YAML.
- `application/json` for JSON.
- If the serialization is not declared, the default is assumed to be YAML.
- If the both YAML and JSON are declared, the default is assumed to be YAML.

DRAFT – please do not distribute

```
POST /offers/2c89f536-3fff-48f7-943f-bcc5c3225be7 HTTP/1.1
Host: foo.example.org
Content-Type: application/yaml

offer:
  ident: "2c89f536-3fff-48f7-943f-bcc5c3225be7"
  status: 'ACCEPTED'
```

If the update is successful, the content of the response is the updated state of the `offer`. The serialization of the response is determined by the `Accept` header in the HTTP request using the same rules as the `GET` method.

```
HTTP/1.1 200 OK
Content-Type: application/yaml; charset=utf-8

offer:
  ident: "2c89f536-3fff-48f7-943f-bcc5c3225be7"
  status: 'ACCEPTED'
  expires: "2023-09-18T07:05:21"
  request:
    executable:
      ....
    resources:
      ....
```

If the update is not successful, the content of the response ... TBD ...

DRAFT – please do not distribute

6 The data model

The following section describes the core data model using a YAML syntax to define the structure with comments to describe the elements.

Note that this does not tie the *ExecutionBroker* specification to a YAML representation. The intention is to define a portable data model that can be serialised into a number of different formats, including YAML, JSON and XML.

6.1 Type URIs and specifications

Several parts of the IVOA *Execution Broker* data model follow a common pattern, using a URI to identify a `type` followed a `spec` section to fill in the details. This pattern is adopted from a similar pattern used by Kubernetes.

```
# A URI to identify the type of thing.
type: "uri:type-a"

# Type specific details about the thing.
spec:
  name: "...."
  size: "...."
  ....
```

However, the *Execution Broker* data model specifically recomends using a long lasting resolvable HTTP URL as the type identifier.

```
# A URL to identify the type of thing.
type: "http://www.example.com/type-a"

# Type specific details about the thing.
spec:
  name: "...."
  size: "...."
  ....
```

The reasoning behind this is that if a service provider notices several clients are requesting *'encabulation'* on their compute nodes, the service provider may not know what this means. However, if we use a resolvable URL to identify the type, a service provider can use it to find a human readable description[19] and decide whether they want to provide it on their platform.

To the web service implementation, both of these requests are just a simple string comparison against a dictionary of known terms:

```
Request  - Can you provide 'encabulation' ?
Response - YES|NO
```

[19] https://tinyurl.com/encabulation

DRAFT – please do not distribute

```
Request  - Can you provide 'https://tinyurl.com/encabulation'
    ?
Response - YES|NO
```

The second form provides a simple mechanism for linking the type identifier to a human readable description.

While it is possible to use IVOA registry URIs to identify types, this specification recomends using a simple HTTP URLs. This lowers the barrier to entry and makes it simpler for the end user to resolve the URL into a description.

6.2 *ExecutionBroker* request

The main section of the *ExecutionBroker* data model describes the `request` block, which describes an *executable thing* and the resources needed to execute it.

The `request` block is used in several places in the data model:

- The main body of an *ExecutionBroker* `request`.
- The main body of each `offer` in an *ExecutionBroker* response.
- The main body of an **ExecutionWorker** *session* record.

6.2.1 Executable

The `executable` section of the `request` block describes the *executable thing*. The core data model simply defines this section as containing a URI identifying the `type` of executable, and a `spec` placeholder for the type specific details.

```
request:
  executable:
    # A URI identifying the executable type.
    type: !!str
    # Type specific details of the executable type.
    spec:
      ....
```

The content of the `spec` section depends on the `type` of executable. The core data model defines 4 types of *executable*; Jupyter and Zeppelin notebooks, and OCI (Docker) and Singularity containers. The structure is designed to make it easy for 3rd parties to extend the data model by defining their own types of *executable*.

DRAFT – please do not distribute

6.2.2 Jupyter notebook

The data model extension for a Jupyter notebook defines the URL to identify the `type`:

```
https://www.purl.org/ivoa.net/executable-types/jupyter-
    notebook
```

The data model extension for a Jupyter notebook contains:

- A URL to download the `notebook` from.
- An optional URL to download a `requirements` file that lists any additional Python modules needed by the notebook.
  - The syntax of the `requirements` file is defined by the `pip` Requirements File Format[20].
  - The URL for the `requirements` file may be an absolute URL, or a relative URL based on the notebook URL.
- An `endpoint` URL that points to the live notebook platform.
  - The `endpoint` URL is set by the **ExecutionWorker** once the *session* is `READY`.

```
request:
  executable:
    # A URL identifying the executable type.
    type: "https://www.purl.org/ivoa.net/executable-types/
   jupyter-notebook"
    # Type specific details of the executable type.
    spec:
      # A URL pointing to the notebook source code.
      notebook: !!str
      # A URL pointing to an optional requirements file.
      requirements: !!str
      # A URL pointing to the live notebook interface.
      endpoint: !!str
```

6.2.3 Zeppelin notebook

The data model extension for a Zeppelin notebook defines the URL to identify the `type`:

```
https://www.purl.org/ivoa.net/executable-types/zeppelin-
    notebook
```

[20] https://pip.pypa.io/en/stable/reference/requirements-file-format/

DRAFT – please do not distribute

The data model extension for a Zeppelin notebook contains:

- A URL to download the `notebook` from.
- An optional URL to download a `requirements` file that lists any additional Python modules needed by the notebook.
  - The syntax of the `requirements` file is defined by the `pip` Requirements File Format[21].
  - The URL for the `requirements` file may be an absolute URL, or a relative URL based on the notebook URL.
- An `endpoint` URL that points to the live notebook platform.
  - The `endpoint` URL is set by the **ExecutionWorker** once the *session* is `READY`.

```
request:
  executable:
    # A URL identifying the executable type.
    type: "https://www.purl.org/ivoa.net/executable-types/
   zeppelin-notebook"
    # Type specific details of the executable type.
    spec:
      # A URL pointing to the notebook source code.
      notebook: !!str
      # A URL pointing to an optional requirements file
      requirements: !!str
      # A URL pointing to the live notebook interface.
      endpoint: !!str
```

6.2.4 Docker container

The data model extension for an Docker container defines the URL to identify the `type`:

```
https://www.purl.org/ivoa.net/executable-types/docker-
   container
```

The data model extension for an Docker container contains:

- The `repo` endpoint address of the repository to download the container image from.
- The name of the `image` to download.

[21] https://pip.pypa.io/en/stable/reference/requirements-file-format/

DRAFT – please do not distribute

- The `tag` identifier to specify which version to download.
- The `digest` identifier to specify which version to download.
- An optional list of `platform` elements which contain:
  - The operating system, `os`, that the container is built for (default `linux`).
  - The system `architecture` that the container is built for (default `amd64`).

```
request:
  executable:
    # A URL identifying the executable type.
    type: "https://www.purl.org/ivoa.net/executable-types/oci
   -container"
    # Type specific details of the executable type.
    spec:
      # The repository to download the image from.
      repo: !!str
      # The name of the container image.
      image: !!str
      # The image tag to specify the version.
      tag: !!str
      # The image digest to specify the version.
      digest: !!str
      # Details of the platform(s) that the container is
   built for.
      platforms:
      - # The system architecture the container is built for.
        architecture: !!str # default: "amd64".
        # The operating system the container is built for.
        os: !!str # default: "linux".
```

6.2.5 Singularity container

The data model extension for a Singularity container defines the URL to identify the `type`:

```
https://www.purl.org/ivoa.net/executable-types/singularity-
   container
```

The data model extension for a Singularity contains:

- ....

DRAFT – please do not distribute

```
request:
  executable:
    # A URL identifying the executable type.
    type: "https://www.purl.org/ivoa.net/executable-types/oci
   -container"
    # Type specific details of the executable type.
    spec:
      ....
      ....
```

6.2.6 Resources

The `resources` section of the `request` block describes the resources needed to execute the task.

The `resources` section contains two lists of resources, one for `compute` resources and one for `storage` resources. Separating these into two lists enables many-to-many relationships between the `compute` and `storage` resources.

```
request:
  # Details of the executable
  executable:
    ....
    ....
  # Details of the resources
  resources:
    # List of compute resources
    compute:
      ....
      ....
    # List of storage resources
    storage:
      ....
      ....
```

6.2.7 Compute resources

The data model for a `compute` resource contains:

- A `name` for the resource, unique within the `request`.
- An optional count of how many `replicas` of the resource are needed.
- A URL identifying the `type` of resource.
- A `spec` placeholder for the type specific details.

DRAFT – please do not distribute

```
request:
  resources:
    compute:
    - # A unique name for this compute resource.
      name: !!str
      # A count of how many replicas are needed.
      replicas: !!int # default: 1
      # A URI identifying the compute resource type.
      type: !!str
      # Type specific details of the compute resource.
      spec:
        ....
        ....
```

The data model structure is designed to make it easy for 3rd parties to extend the data model by defining their own types of computing resources.

6.2.8 Generic compute resources

The core data model defines a generic computing resource that has cpu cores, memory and a hierarchical filesystem. This represents a common abstraction of a virtual machine, an OCI container runtime or a Kubernetes Pod.

The data model for a generic computing resource defines the URL to identify the type:

```
https://www.purl.org/ivoa.net/resource-types/generic-compute
```

The `spec` for a generic computing resource may contain the following optional properties:

- The minimum number of cpu cores needed, set by the client in the *ExecutionBroker* `request`.
- The maximum number of cpu cores available, set by the *ExecutionBroker service* in an `offer`.
- The minimum amount of memory needed (in GiB), set by the client in the *ExecutionBroker* `request`.
- The maximum amount of memory available (in GiB), set by the *ExecutionBroker service* in an `offer`.

```
request:
  resources:
    compute:
    - # A unique name for this compute resource.
      name: !!str
      # A count of how many replicas are needed.
```

DRAFT – please do not distribute

```
      replicas: !!int # default: 1
      # A URL identifying the compute resource type.
      type: "https://www.purl.org/ivoa.net/resource-types/
   generic-compute"
      spec:
        # The minimum and maximum number of cpu cores needed.
        cores:
          min: !!float # default: 1
          max: !!float # default: cores.min
        # The minimum and maximum amount of memory needed, in
    GiB.
        memory:
          min: !!float # default: 1
          max: !!float # default: memory.min
        ....
```

The `spec` for a generic computing resource may contain a list of `volumes` that describe how a storage resource should be mounted in the compute resource's filesystem.

Each `volume` element contains:

- A `name` for the volume, unique within this `request`.
- A reference (by name) to the corresponding storage `resource`.
- The `path` that the volume should be mounted at in the compute resource's filesystem.
- The read write access mode ('`rw`' for read and write, or '`ro`' for read only).

```
request:
  resources:
    compute:
    - # A unique name for this compute resource.
      name: !!str
      # A count of how many replicas are needed.
      replicas: !!int # default: 1
      # A URL identifying the compute resource type.
      type: "https://www.purl.org/ivoa.net/resource-types/
   generic-compute"
      spec:
        ....
        # A list of volume mounts.
        volumes:
        - # A unique name for this volume.
          name: !!str
          # The name of the storage resource this volume is
   linked to.
```

DRAFT – please do not distribute

```
          resource: !!str
          # The path in the compute resource's filesystem.
          path: !!str
          # The access mode, 'rw' or 'ro'.
          mode: !!str
```

6.2.9 Storage resources

```
....
....
```

6.2.10 Authentication

```
....
....
```

6.2.11 DateTime

```
....
....
```

6.3 Response

6.3.1 Positive response

A positive (yes, this platform can execute the task) *ExecutionBroker service* response contains `result` set to `YES`, and a list of *ExecutionBroker offers*.

6.3.2 Offer

Each *ExecutionBroker offer* contains a unique identifier a status value and an expiry time for the *ExecutionBroker offer* followed by an updated copy of the `request` with details filled in by the *ExecutionBroker service*.

The `offer status` ....

```
response:
  # The top level result.
  result: !!str 'YES'
  # A list of offers to match the request.
  offers:
  - # Unique identifier for this offer.
    ident:  !!str
    # Status of this offer.
    status: !!str # [OFFERED, ACCEPTED, REJECTED, EXPIRED]
    # Expiry date for this offer.
```

DRAFT – please do not distribute

```
    expires: !!str # ISO 8601 date time
    # An updated copy of the request body.
    request:
      executable:
        ....
      resources:
        ....
      datetime:
        ....
```

6.3.3 Negative response

A negative (no, this platform cannot execute the task) *ExecutionBroker service* response contains `result` set to `NO`, followed by an optional list of reasons explaining why the request was rejected.

Each `reason` consists of a data model path to an element in the original `request` and a text message explaining why the element was rejected.

```
response:
  # The top level result.
  result: !!str 'NO'
  # A list of reasons explaining why the request was rejected
    .
  reasons:
  - # The path to an element in the request body
    path: !!str
    # A text message why the element was rejected.
    text: !!str
```

6.4 Session

An **ExecutionWorker** *session* record consists of a unique identifier for the *session*, and a `phase` indicating what execution stage the *session* has reached. Followed by an updated copy of the request `body` from the *ExecutionBroker ExecutionBroker offer* with details filled in by the **ExecutionWorker** as the *session* is executed.

```
job:
  # A unique identifier for this session
  ident: !!str
  # The current execution phase.
  phase: !!str
  # An updated copy of the request body
  request:
    executable:
      ....
    resources:
      ....
```

DRAFT – please do not distribute

```
    datetime:
      ....
```

6.5 Extensions

```
....
....
```

6.5.1 nvidia gpu

```
....
....
```

6.5.2 Xilinx FPGA

```
type: "https://www.purl.org/ivoa.net/resource-types/xilinx-
    vu19p"
# The details, specific to a 'Xilinx FPGA'.
....
....
```

DRAFT – please do not distribute

A Changes from Previous Versions

No previous versions yet.

DRAFT – please do not distribute

A Resource type URLs

https://www.purl.org/ivoa.net/executable-types/example https://github.com/ivoa-std/ExecutionBroker/blob/main/types/executable-types/example-executable.md

https://www.purl.org/ivoa.net/executable-types/jupyter-notebook https://github.com/ivoa-std/ExecutionBroker/blob/main/types/executable-types/jupyter-notebook.md

https://www.purl.org/ivoa.net/executable-types/oci-container https://github.com/ivoa-std/ExecutionBroker/blob/main/types/executable-types/oci-container.md

https://www.purl.org/ivoa.net/resource-types/generic-compute https://github.com/ivoa-std/ExecutionBroker/blob/main/types/resource-types/generic-compute.md

https://www.purl.org/ivoa.net/resource-types/nvidia-ad104 https://github.com/ivoa-std/ExecutionBroker/blob/main/types/resource-types/nvidia-ad104.md

https://www.purl.org/ivoa.net/resource-types/xilinx-vu19p https://github.com/ivoa-std/ExecutionBroker/blob/main/types/resource-types/xilinx-vu19p.md

https://www.purl.org/ivoa.net/storage-types/ephemeral-storage https://github.com/ivoa-std/ExecutionBroker/blob/main/types/storage-types/ephemeral-storage.md

https://www.purl.org/ivoa.net/storage-types/amazon-s3 https://github.com/ivoa-std/ExecutionBroker/blob/main/types/storage-types/amazon-s3.md

https://www.purl.org/ivoa.net/storage-types/persistent-storage https://github.com/ivoa-std/ExecutionBroker/blob/main/types/storage-types/persistent-storage.md

https://www.purl.org/ivoa.net/storage-types/rucio-storage https://github.com/ivoa-std/ExecutionBroker/blob/main/types/storage-types/rucio-storage.md

https://www.purl.org/ivoa.net/storage-types/vospace-storage https://github.com/ivoa-std/ExecutionBroker/blob/main/types/storage-types/vospace-storage.md

https://www.purl.org/ivoa.net/authentication-types/basic-auth https://github.com/ivoa-std/ExecutionBroker/blob/main/types/authentication-types/basic-auth.md

https://www.purl.org/ivoa.net/authentication-types/oidc-auth https://github.com/ivoa-std/ExecutionBroker/blob/main/types/authentication-types/oidc-auth.md

https://www.purl.org/ivoa.net/authentication-types/rfc7519-token https://github.com/ivoa-std/ExecutionBroker/blob/main/types/authentication-types/rfc7519-token.md

https://www.purl.org/ivoa.net/webservice-types/value-update https://github.com/ivoa-std/ExecutionBroker/blob/main/types/webservice-types/value-update.md

YAML specification https://yaml.org/spec/

YAML timestamp https://yaml.org/type/timestamp.html

References

Arviset, C., Gaudet, S. and IVOA Technical Coordination Group (2010), 'IVOA Architecture Version 1.0', IVOA Note 23 November 2010. http://doi.org/10.5479/ADS/bib/2010ivoa.rept.1123A

DRAFT – please do not distribute

Bradner, S. (1997), 'Key words for use in RFCs to indicate requirement levels', RFC 2119.
http://www.ietf.org/rfc/rfc2119.txt

Graham, M., Morris, D. and Rixon, G. (2009), 'VOSpace specification Version 1.15', IVOA Recommendation 07 October 2009.
http://doi.org/10.5479/ADS/bib/2009ivoa.specQ1007G

International Organization for Standardization (2004), 'Data elements and interchange formats – information interchange – representation of dates and times'.
http://www.iso.org/iso/catalogue_detail?csnumber=40874

Plante, R., Graham, M., Rixon, G. and Taffoni, G. (2010), 'IVOA Credential Delegation Protocol Version 1.0', IVOA Recommendation 18 February 2010, arXiv:1110.0509.
http://doi.org/10.5479/ADS/bib/2010ivoa.spec.0218P

Taffoni, G., Schaaf, A., Rixon, G. and Major, B. (2017), 'SSO - Single-Sign-On Profile: Authentication Mechanisms Version 2.0', IVOA Recommendation 24 May 2017, arXiv:1709.00171.
http://doi.org/10.5479/ADS/bib/2017ivoa.spec.0524T